

Resumo Completo: Capítulo 6 - Impasses (Deadlocks)

1.0 Introdução aos Impasses

1.1 O que é um Impasse (Deadlock)?

Um impasse, ou deadlock, é uma situação em que um conjunto de dois ou mais processos fica permanentemente bloqueado, pois cada processo do conjunto está esperando por um evento (tipicamente a liberação de um recurso) que só pode ser causado por outro processo do mesmo conjunto. Como todos estão esperando, nenhum deles pode prosseguir.

- **Definição Formal:** Um conjunto de processos está em situação de impasse se cada processo no conjunto estiver esperando por um evento que apenas outro processo no conjunto pode causar.
- **Exemplo Clássico:**
 1. O **Processo A** solicita e obtém acesso exclusivo a um **scanner**.
 2. O **Processo B** solicita e obtém acesso exclusivo a um **gravador de Blu-ray**.
 3. O **Processo A** agora solicita o **gravador de Blu-ray**, mas é bloqueado, pois B o possui.
 4. O **Processo B** agora solicita o **scanner**, mas é bloqueado, pois A o possui.
 - *Resultado:* Ambos os processos A e B ficarão bloqueados para sempre, esperando um pelo outro.

Impasses não se limitam a um único sistema, podendo ocorrer em redes (recursos remotos) e em bancos de dados (bloqueio de registros).

1.2 Recursos

Para generalizar a discussão, os objetos pelos quais os processos competem são chamados de **recursos**.

- **Definição de Recurso:** Qualquer coisa que precise ser adquirida, usada e liberada com o passar do tempo, e que só pode ser usada por um processo de cada vez.
 - **Recursos de Hardware:** Impressoras, scanners, unidades de fita, gravadores de DVD.
 - **Recursos de Software:** Registros em um banco de dados, entradas em tabelas internas do sistema, arquivos.

1.2.1 Classificação de Recursos

1. Recursos Preemptíveis:

- São recursos que podem ser tomados de um processo proprietário sem causar efeitos adversos ou falhas.
- **Exemplo Principal:** A memória principal. Um processo pode ter suas páginas de memória salvas em disco (swap out) para liberar memória para outro processo, e depois ser restaurado (swap in) sem perda de dados.
- Impasses envolvendo recursos preemptíveis geralmente podem ser resolvidos pela realocação do recurso.

2. Recursos Não Preemptíveis:

- São recursos que não podem ser tomados à força do processo que os detém sem causar uma falha ou corromper o trabalho.
- **Exemplo Principal:** Um gravador de Blu-ray. Se o gravador for tomado no meio de uma gravação, o disco será corrompido.
- **A grande maioria dos impasses envolve recursos não preemptíveis.**

1.2.2 Sequência de Uso de Recursos

O uso de um recurso por um processo segue uma sequência abstrata de três passos:

1. **Solicitar o recurso:** Se o recurso não estiver disponível, o processo solicitante é bloqueado (forçado a esperar).
 2. **Usar o recurso:** O processo opera sobre o recurso.
 3. **Liberar o recurso:** O processo torna o recurso disponível para outros processos.
-

2.0 Condições para Ocorrência de Impasses de Recurso

Conforme demonstrado por Coffman et al. (1971), quatro condições **devem ser válidas simultaneamente** para que um impasse de recurso ocorra.

1. **Condição de Exclusão Mútua:** Cada recurso está alocado a no máximo um processo ou está disponível. Nenhum recurso pode ser compartilhado simultaneamente.
2. **Condição de Posse e Espera (Hold and Wait):** Processos que já detêm recursos podem solicitar novos recursos, e permanecem com os seus enquanto esperam.
3. **Condição de Não Preempção:** Recursos já concedidos a um processo não podem ser tomados à força; eles devem ser liberados voluntariamente pelo processo que os detém.
4. **Condição de Espera Circular:** Deve existir uma cadeia circular de dois ou mais processos, onde cada processo está esperando por um recurso que é detido pelo próximo membro da cadeia.

Importante: A ausência de qualquer uma dessas quatro condições torna impossível a ocorrência de um impasse de recurso.

3.0 Modelagem de Impasses com Grafos de Alocação de Recursos

Os impasses podem ser modelados visualmente usando **grafos de alocação de recursos**, que são grafos dirigidos com dois tipos de nós:

- **Círculos:** Representam os **processos**.
- **Quadrados:** Representam as **classes de recursos**.

As arestas (setas) têm os seguintes significados:

- **Seta do Recurso para o Processo ($R \rightarrow P$):** Indica que o recurso R foi **alocado** ao processo P .
- **Seta do Processo para o Recurso ($P \rightarrow R$):** Indica que o processo P está **solicitando/esperando** pelo recurso R .

Regra Fundamental: Um **ciclo** no grafo de alocação de recursos indica a existência de um impasse, assumindo que há apenas uma instância de cada recurso no ciclo.

- **Exemplo de Impasse no Grafo:** $C \rightarrow T \rightarrow D \rightarrow U \rightarrow C$
 - O processo C espera pelo recurso T.
 - O recurso T está alocado ao processo D.
 - O processo D espera pelo recurso U.
 - O recurso U está alocado ao processo C.
-

4.0 Estratégias para Lidar com Impasses

Existem quatro estratégias principais para tratar os impasses:

1. **Ignorar o Problema (Algoritmo do Avestruz):** A estratégia mais comum em sistemas operacionais de propósito geral (como Windows e UNIX). Assume-se que impasses são tão raros que o custo de detectá-los ou preveni-los é maior do que o prejuízo de reinicializar o sistema quando um ocorre.
 2. **Deteção e Recuperação:** Permitir que os impasses ocorram, detectá-los quando acontecerem e tomar ações para corrigir o sistema.
 3. **Evitação Dinâmica:** Alocar recursos de forma cuidadosa, com base em informações sobre o uso futuro de recursos, para garantir que o sistema nunca entre em um estado de impasse.
 4. **Prevenção Estrutural:** Negar estruturalmente uma das quatro condições de Coffman, tornando os impasses impossíveis por projeto.
-

5.0 Deteção e Recuperação de Impasses

5.1 Deteção com um Recurso de Cada Tipo

Se cada classe de recurso possui apenas uma instância, a deteção de impasse se resume a encontrar ciclos no grafo de alocação de recursos. Isso pode ser feito com algoritmos de busca em profundidade (DFS).

5.2 Deteção com Múltiplos Recursos de Cada Tipo (Algoritmo Matricial)

Quando existem múltiplas instâncias de cada recurso, um ciclo no grafo não garante um impasse. Um algoritmo baseado em matrizes é necessário.

- **Estruturas de Dados:**
 - **E:** Vetor de recursos **Existentes** (total de instâncias de cada tipo). Ex: $E = (4 \ 2 \ 3 \ 1)$ -> 4 fitas, 2 plotters, 3 scanners, 1 Blu-ray.
 - **A:** Vetor de recursos **Disponíveis** (Available).
 - **C:** Matriz de alocação **Atual** (Current). $C[i][j]$ é o nº de instâncias do recurso j alocadas ao processo i .
 - **R:** Matriz de **Requisição**. $R[i][j]$ é o nº de instâncias do recurso j que o processo i ainda precisa.

- **Algoritmo de Detecção:**

1. Marque todos os processos como "não terminados".
2. Procure por um processo P_i não terminado cuja linha de requisição R_i seja menor ou igual ao vetor de recursos disponíveis A (ou seja, $R_i[j] \leq A[j]$ para todos os j).
3. Se nenhum processo assim for encontrado, o algoritmo termina. Os processos não terminados estão em impasse.
4. Se tal processo P_i for encontrado, ele pode (hipoteticamente) terminar. Libere seus recursos: $A = A + C_i$ (some a linha i da matriz C ao vetor A). Marque P_i como "terminado".
5. Volte para o passo 2.

- **Exemplo (Figura 6.7 do livro):**

- $E = (4 \ 2 \ 3 \ 1)$, $A = (2 \ 1 \ 0 \ 0)$
- Processo 1 requisita $(2 \ 0 \ 0 \ 1)$, não pode ser atendido (A não tem Blu-ray).
- Processo 2 requisita $(1 \ 0 \ 1 \ 0)$, não pode ser atendido (A não tem scanner).
- Processo 3 requisita $(2 \ 1 \ 0 \ 0)$, **pode ser atendido!**
- O P3 "termina" e libera seus recursos. Novo $A = (2 \ 1 \ 0 \ 0) + (0 \ 1 \ 2 \ 0) = (2 \ 2 \ 2 \ 0)$.
- Agora, com o novo A , o P2 pode ser atendido. Ele "termina". Novo $A = (2 \ 2 \ 2 \ 0) + (2 \ 0 \ 0 \ 1) = (4 \ 2 \ 2 \ 1)$.
- Finalmente, o P1 pode ser atendido.
- **Conclusão:** Todos os processos podem terminar. Não há impasse no sistema.

5.3 Recuperação de um Impasse

Uma vez detectado, o impasse deve ser desfeito.

1. **Recuperação por Preempção:** Tomar um recurso à força de um processo e dá-lo a outro. É muito difícil de implementar sem causar inconsistências.
2. **Recuperação por Retrocesso (Rollback):** Fazer com que os processos retornem a um estado anterior seguro (checkpoint) e reiniciem a partir dali. Isso requer que o sistema salve periodicamente o estado dos processos.
3. **Recuperação por Eliminação de Processos:** A forma mais bruta e simples. O sistema mata um ou mais processos envolvidos no ciclo de impasse. A escolha de qual processo matar pode ser baseada em prioridade, tempo de execução, etc.

6.0 Evitação de Impasses

A evitação de impasses busca tomar decisões de alocação que não levem a um estado de impasse. Requer conhecimento prévio das necessidades máximas de recursos dos processos.

6.1 Estados Seguros e Inseguros

- **Estado Seguro:** É um estado no qual existe **pelo menos uma sequência de alocação** que permite que todos os processos terminem. O sistema pode garantir que não haverá impasse.

- **Estado Inseguro:** É um estado do qual **não há garantia** de que todos os processos terminarão. Um estado inseguro **não é** um impasse, mas *pode* levar a um.

O objetivo dos algoritmos de evitação é garantir que o sistema permaneça sempre em um estado seguro.

6.2 O Algoritmo do Banqueiro

Proposto por Dijkstra, este algoritmo verifica se a concessão de um recurso a um processo levará o sistema a um estado seguro. Se sim, o recurso é alocado; se não, o processo é bloqueado.

- **Requisito:** Cada processo deve declarar sua necessidade **máxima** de recursos ao ser criado.
 - **Funcionamento:**
 1. Quando um processo solicita um recurso, o sistema primeiro verifica se há recursos disponíveis para atendê-lo.
 2. Se houver, o sistema **simula** a alocação do recurso.
 3. Em seguida, ele executa um algoritmo (similar ao de detecção) para verificar se o novo estado resultante é **seguro**. Ele procura uma sequência de processos que possam terminar, dado o novo estado de alocação.
 4. Se o estado for seguro, a alocação é efetivada. Se for inseguro, a alocação é negada e o processo é suspenso.
 - **Limitações Práticas:** É raramente usado na prática porque os processos geralmente não sabem suas necessidades máximas de recursos com antecedência, e o número de processos e recursos no sistema é dinâmico.
-

7.0 Prevenção de Impasses

A prevenção consiste em projetar o sistema de forma a violar (negar) uma das quatro condições de Coffman.

1. Atacando a Exclusão Mútua:

- Permitir o compartilhamento de recursos. Impossível para recursos intrinsecamente não compartilháveis (ex: impressora).
- **Técnica:** Spooling. Vários processos podem "imprimir" ao mesmo tempo, mas suas saídas são direcionadas para um arquivo em disco. Um único processo (daemon de impressão) lê esses arquivos e os envia para a impressora real, um de cada vez.

2. Atacando a Posse e Espera:

- **Abordagem 1:** Exigir que um processo solicite **todos** os seus recursos antes de iniciar a execução. Ineficiente, pois os recursos ficam alocados por muito tempo sem uso.
- **Abordagem 2:** Exigir que um processo libere todos os recursos que detém antes de solicitar um novo. Também pode ser ineficiente.

3. Atacando a Não Preempção:

- Permitir que recursos sejam tomados à força (preempção). Viável para recursos como a CPU e a memória, mas muito difícil ou impossível para outros como impressoras ou arquivos em atualização.

4. Atacando a Espera Circular:

- **Abordagem Principal:** Estabelecer uma **ordenação numérica global** para todos os recursos. A regra é que os processos só podem solicitar recursos em ordem crescente.
 - **Exemplo:** Se Impressora=1, Fita=2, Plotter=3, um processo pode solicitar a Impressora e depois a Fita. Mas se ele já possui a Fita (recurso 2), não pode solicitar a Impressora (recurso 1).
 - Isso garante que um grafo de alocação de recursos nunca terá um ciclo, prevenindo impasses de forma eficaz. A dificuldade está em encontrar uma ordenação prática para todos os tipos de recursos de um sistema.
-

8.0 Outras Questões Relacionadas

8.1 Livelock

Similar a um impasse, pois os processos não fazem progresso. No entanto, em um livelock, os processos **não estão bloqueados** – eles estão constantemente mudando de estado em resposta uns aos outros.

- **Analogia:** Duas pessoas se encontram em um corredor e ambas tentam sair do caminho dando um passo para o mesmo lado, repetidamente. Elas estão ativas, mas sem progresso.

8.2 Inanição (Starvation)

Ocorre quando uma política de alocação de recursos, embora pareça justa, pode fazer com que um processo espere indefinidamente por um recurso.

- **Exemplo:** Em uma fila de impressão que sempre prioriza o trabalho mais curto, um processo com um trabalho de impressão muito longo pode nunca ser atendido se houver um fluxo constante de trabalhos curtos.
- **Diferença para o Impasse:** Em inanição, o processo não está necessariamente bloqueado por outro processo que detém o recurso que ele precisa; ele apenas é constantemente preterido pelo escalonador.

8.3 Impasses de Comunicação

Ocorrem quando a comunicação entre processos falha.

- **Exemplo:** O Processo A envia uma mensagem para B e bloqueia esperando uma resposta. A mensagem se perde na rede. O Processo B está bloqueado esperando por uma mensagem de A. Ambos esperam para sempre.
- **Solução Comum:** Uso de **timeouts**. Se uma resposta não chegar dentro de um certo tempo, o Processo A assume que a mensagem foi perdida e a reenvia.